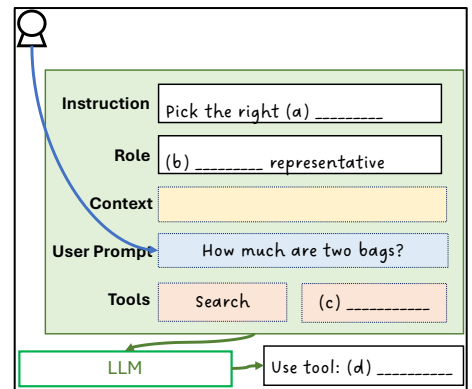
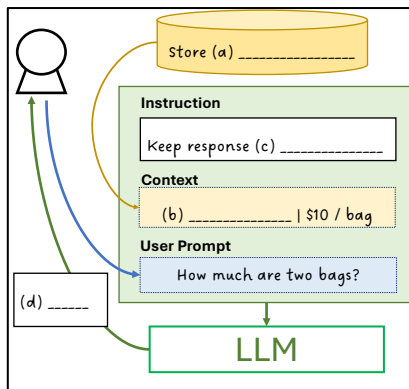
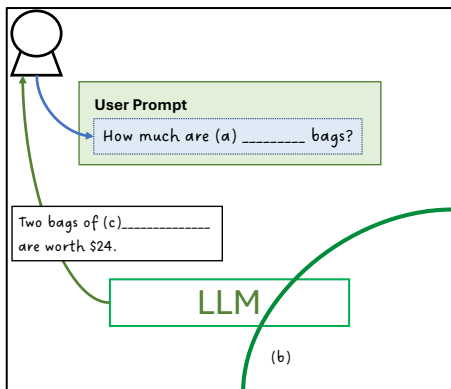


LLM vs RAG vs Agent Workbook



Prof. Tom Yeh

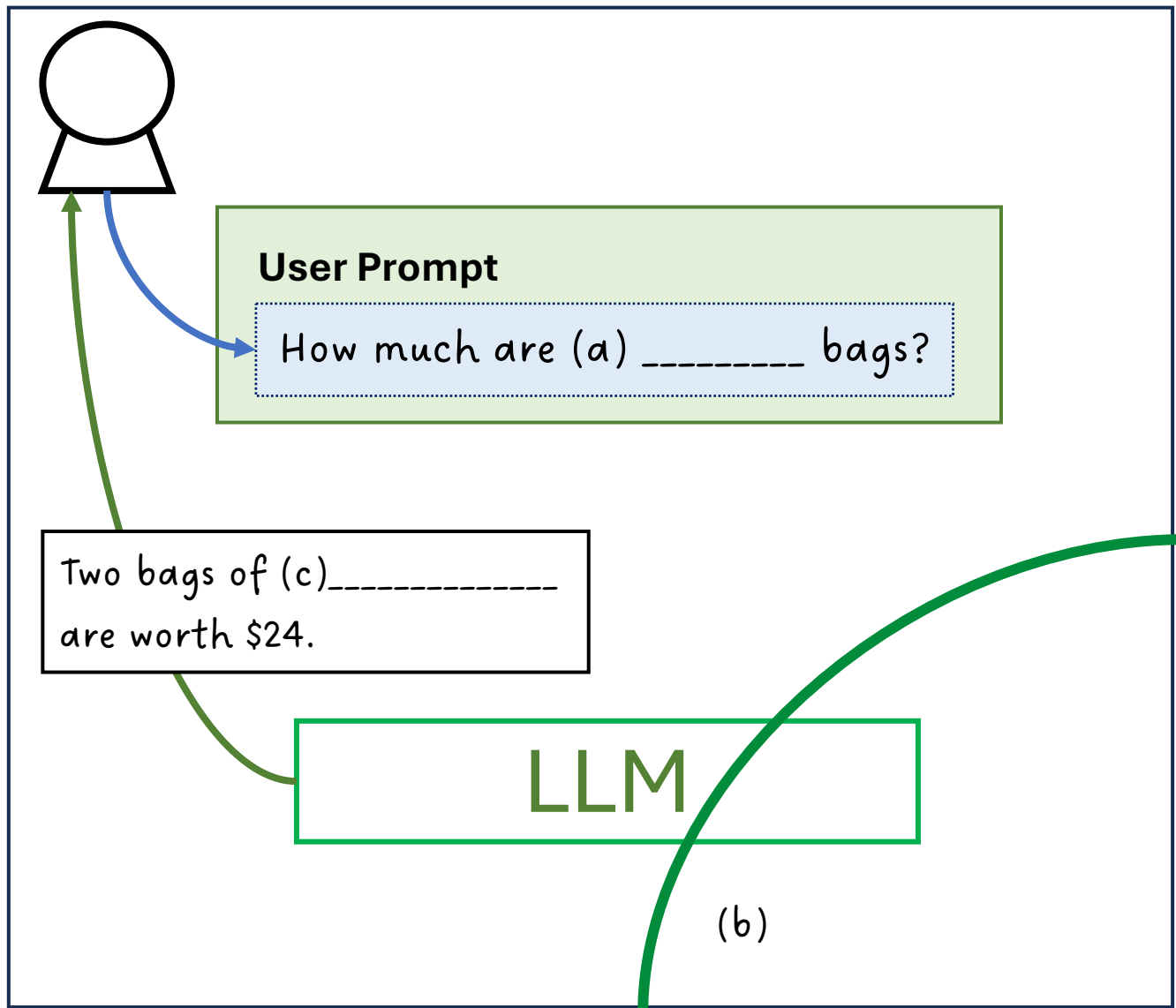
sponsored by



This workbook was made possible through the generous support of **AMESA (Advanced Modular Enterprise Systems of Autonomy)** — a company pioneering the future of enterprise autonomy through an AI agent proving ground and orchestration platform built for real-world performance.

By supporting *AI by Hand* 🖋️, AMESA advances our shared mission to make agentic AI understandable, teachable, and trustworthy — bridging hands-on education with enterprise-grade autonomy.

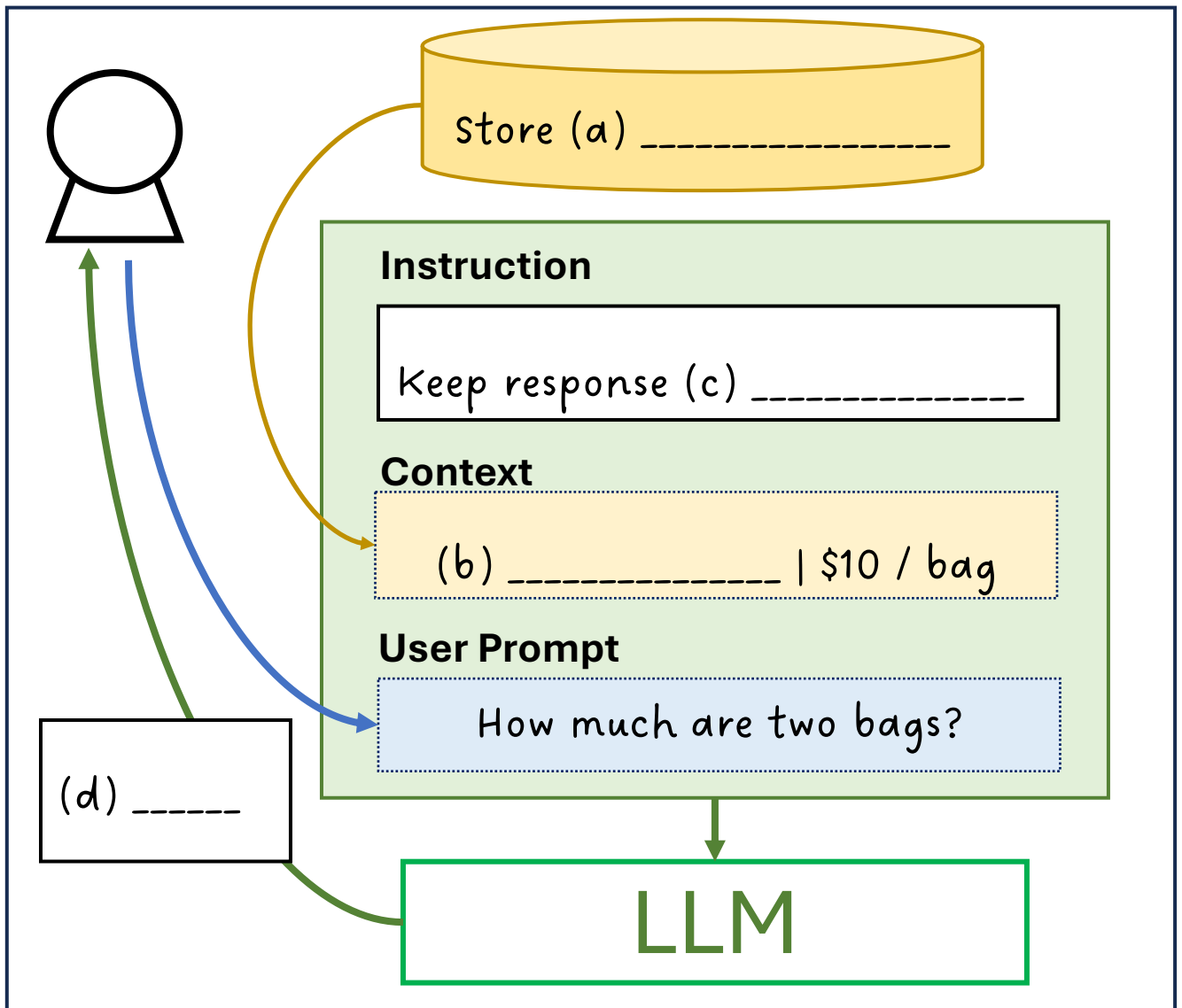
Visit <https://www.amesa.com/> to learn more.



LLM

Let's consider a scenario where the user asks, "How much are (a) two bags?" The LLM, trained on (b) Internet data, tries to infer what the bags might contain. Since it has no access to store-specific information, it makes the most probable guess—apples—and estimates that a bag of apples costs around \$12. Therefore, it responds, "Two bags of (c) apples are worth \$24."

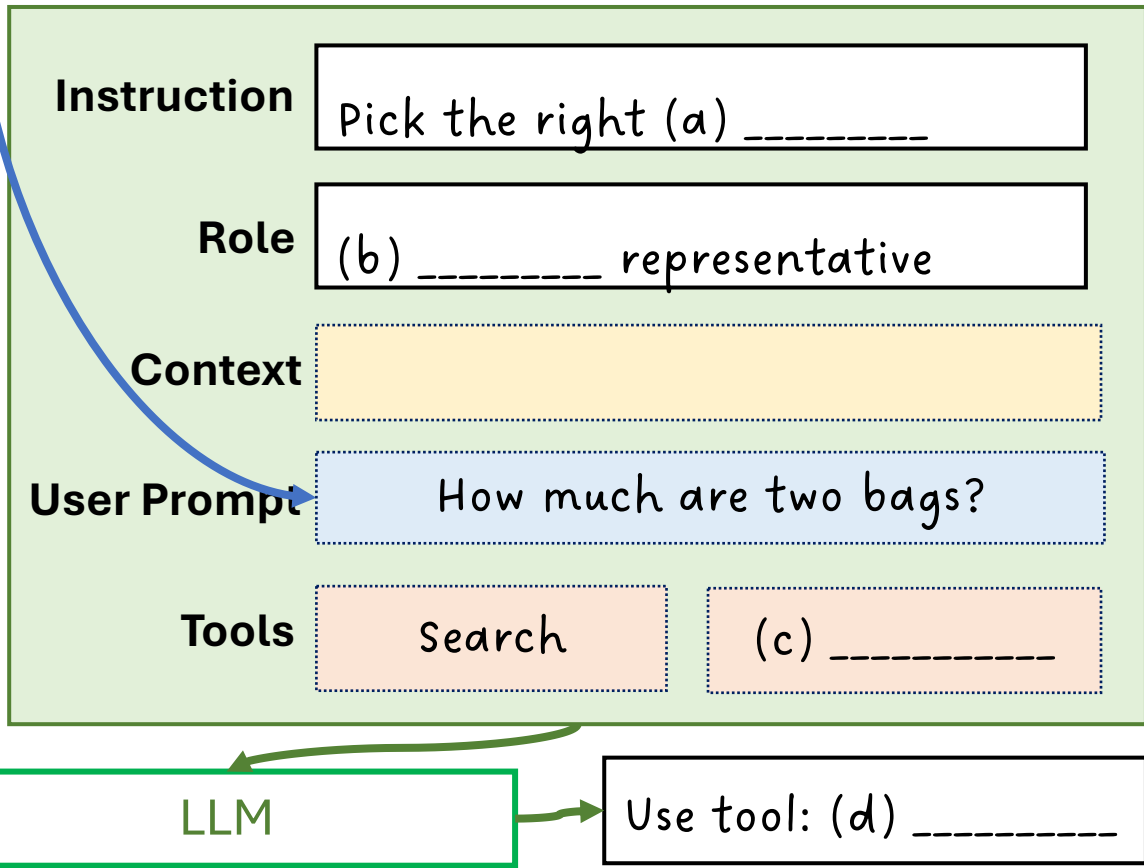
However, if this conversation were taking place in a *baby store*, the LLM would not know that context. It would still assume the bags refer to apples rather than baby products like diapers, resulting in an inaccurate answer.



RAG

The user asked, “How much are two bags?” Before sending the prompt to the LLM, we first checked the store (a) database and retrieved the information that (b) diapers are sold for \$10 per bag. This context information is specific to the store, not in the LLM’s training data, so the LLM would not have known it on its own.

Since we want the LLM to provide only a short answer (without showing its reasoning), we added a system instruction: “keep response (c) brief.” We then augmented the user prompt with this context and instruction before sending it to the LLM. The LLM responded concisely with (d) “\$20.”

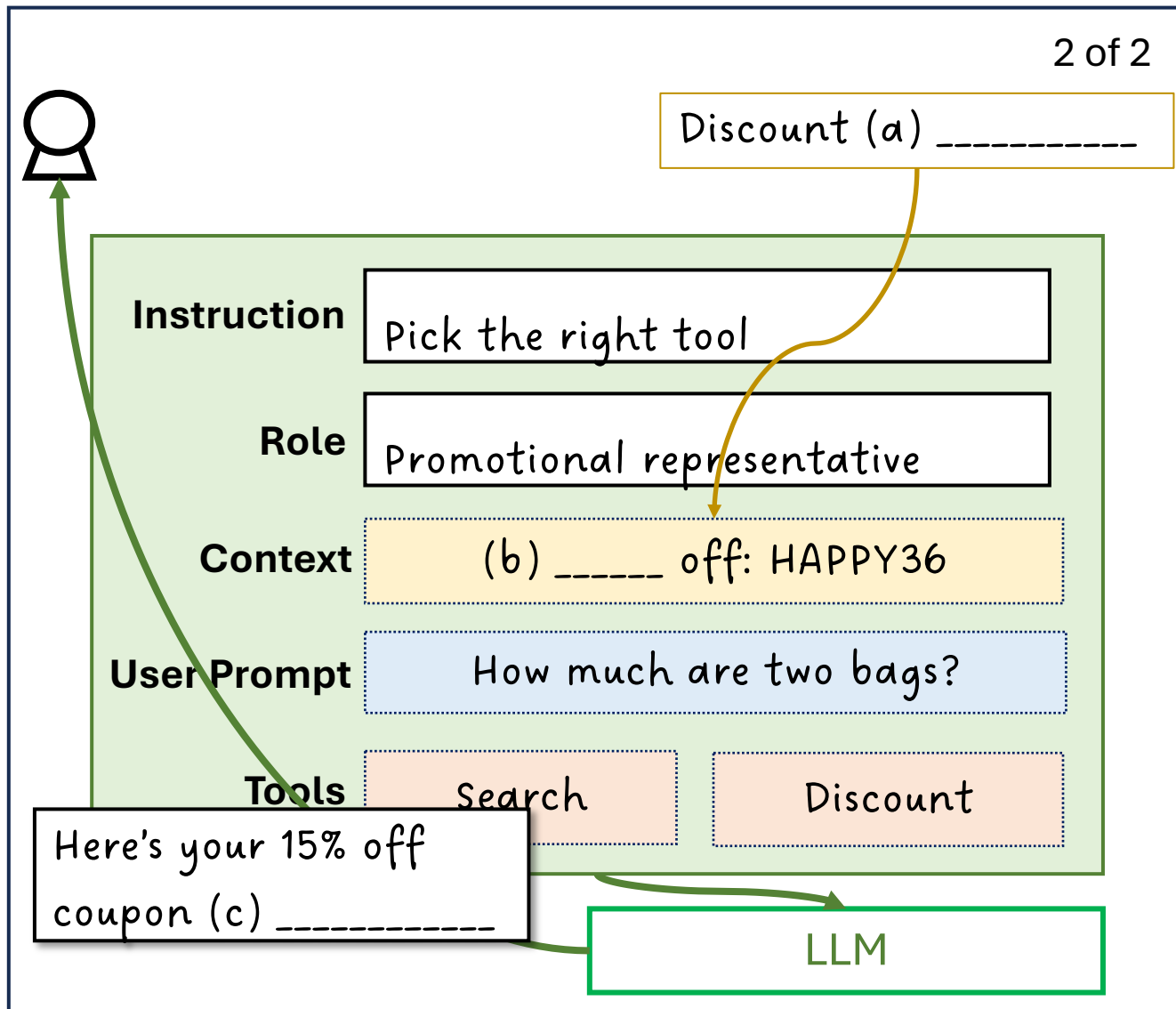


Agent

The user asked, “How much are two bags?” Instead of immediately searching the database, we first want the agent to decide whether a database search is even necessary. After all, database search is just one tool — another possible action could be to offer a (c) Discount that appeals to the customer. We want the agent to intelligently pick the right (a) tool to use.

How should the agent make this selection? Rather than following a set of explicit rules, it is easier to assign a role — for example, that of a (b) *promotional representative*. The LLM, having learned extensively from the Internet, can infer what a promotional representative would typically do given the user’s prompt.

To implement this agentic behavior, we augment the user prompt with the instruction, the assigned role, the context, and the available tools, and send all of this to the LLM. The LLM then responds: “Use tool (d) Discount” — an excellent choice for someone acting as a promotional representative.



Then, having selected the “Discount” tool, the agent calls the Discount (a) Server to generate a new (b) 15% off coupon code: **HAPPY36**. This code is then added to the context and sent, along with the updated instruction, to the LLM — marking the second call to the LLM. This time, the LLM determines that no further tool use is necessary, since it now has all the required information. It generates a message: “Here’s your 15% off code: (c) **HAPPY36**” and displays this to the user.

LLM vs. RAG vs. Agent — Key Differences

LLM relies solely on its *training data* and general world knowledge. It guesses likely context (e.g., “bags of apples”) and responds accordingly, even if the guess may be wrong.

RAG supplements the LLM with *retrieved real-world data* (e.g., diapers are \$10 per bag) so the answer is grounded in facts the model wouldn’t otherwise know.

Agent goes further by *deciding which tool to use*—database, discount, or otherwise—based on its assigned *role* and situation, showing reasoning and control before generating the final response.

	LLM	RAG	Agent
Knowledge Use			
Control Logic			

Knowledge Source

The **LLM** relies entirely on [Internet data] learned during pretraining, using general world knowledge to generate likely answers. **RAG**, by contrast, augments the LLM with [retrieved data]—information pulled from its own or external databases, such as store prices or documents—so its responses are grounded in current and specific facts. **Agent** goes a step further by connecting the LLM to [tools and APIs], enabling it to take actions, access live systems, and dynamically gather the information it needs before responding.

Control Logic

The **LLM** follows a [single-step logic]: it receives a prompt and produces an answer directly, without any intermediate reasoning or actions. **RAG** adds [fetch-then-answer logic], where the system first retrieves relevant information and then passes it to the LLM to generate a grounded response. **Agent** employs [multi-step reasoning logic], deciding which tool to use, when to call it, and how to integrate results—allowing it to plan, act, and respond adaptively across multiple steps.